

UNIVERSIDAD CRISTIANA EVANGELICA NUEVO MILENIO



UCENM

LABORATORIO DE SISTEMAS OPERATIVOS II

NOMBRE DEL DOCENTE:

RICARDO ENRIQUE LAGOS MENDOZA

NOMBRE DEL ESTUDIANTE:

EDUAR JOSUE MORENO BONILLA	120140083
FERNANDO JOSÉ GARCÍA	222020008
JOSÉ RICARDO SALAZAR	122070020
EDAR MURCIA	322450013
ERASMO GASPAR MEJÍA	120160026
BRYAN JOELLE CAMPIGOTTI	122070150
MARCO ANDRE IZAGUIRRE	122079169
OSCAR JONATHAN MONTEJO	121390061

TRABAJO:

EXAMEN I

FECHA DE ENTREGA:

30/06/2026

Índice

1. Introducción	3
2. Objetivos	4
2.1 Objetivo General.....	4
2.2 Objetivos Específicos	4
3. Marco Teórico.....	5
3.1 Docker y la Virtualización mediante Contenedores	5
3.2 Imágenes de Contenedores.....	5
3.3 Contenedores.....	5
3.4 Volúmenes	6
3.5 Administración y Ciclo de Vida del Contenedor	6
3.6 Redes en Docker	6
4. Actividades Realizadas	7
4.1 Preparación del Entorno	7
4.2 Implementación del Servidor Web.....	7
4.3 Publicación del Sitio Web	7
4.4 Administración del Contenedor	8
4.5 Administración General de Docker	8
5. Conclusiones.....	9
6. Anexos	10

1. Introducción

Docker es una plataforma de virtualización a nivel de sistema operativo que permite empaquetar aplicaciones junto con todas sus dependencias en unidades ligeras y portables denominadas contenedores. A diferencia de las máquinas virtuales tradicionales, los contenedores comparten el kernel del sistema operativo anfitrión, lo que los hace más rápidos de iniciar y más eficientes en el uso de recursos.

2. Objetivos

2.1 Objetivo General

Evaluar las competencias prácticas en la administración de contenedores Docker mediante la creación de un entorno funcional, utilizando imágenes oficiales, volúmenes, redes y servidores web, publicando el contenido de una asignatura sin instalar software directamente sobre el sistema operativo.

2.2 Objetivos Específicos

- Verificar el funcionamiento de Docker Desktop y descargar imágenes oficiales desde un repositorio público.
- Implementar un contenedor con un servidor web Nginx funcional, configurado con puertos, ejecución en segundo plano y reinicio automático.
- Publicar un sitio web personalizado mediante el uso de volúmenes, evitando la modificación directa del contenedor.
- Aplicar comandos de administración de contenedores tales como inicio, detención, inspección y eliminación.
- Administrar de forma general los recursos de Docker: imágenes, contenedores, volúmenes y redes.

3. Marco Teórico

3.1 Docker y la Virtualización mediante Contenedores

Docker es una herramienta que automatiza el despliegue de aplicaciones dentro de contenedores, aprovechando las características de aislamiento de procesos del kernel Linux (namespaces y cgroups). Esto permite ejecutar múltiples aplicaciones de forma aislada sobre un mismo sistema operativo, sin la sobrecarga que implica virtualizar hardware completo como ocurre con las máquinas virtuales tradicionales. Docker Desktop es la aplicación de escritorio que integra el motor de Docker, una interfaz gráfica de administración y herramientas complementarias para gestionar contenedores, imágenes, volúmenes y redes de forma local.

3.2 Imágenes de Contenedores

Una imagen es una plantilla inmutable y de solo lectura que contiene el sistema de archivos, las bibliotecas y la configuración necesaria para ejecutar una aplicación. Las imágenes se obtienen habitualmente de repositorios públicos como Docker Hub, donde existen imágenes oficiales mantenidas por los propios desarrolladores del software, como es el caso de Nginx. A partir de una misma imagen pueden crearse múltiples contenedores independientes entre sí.

3.3 Contenedores

Un contenedor es una instancia en ejecución de una imagen. Cada contenedor cuenta con su propio sistema de archivos, procesos y configuración de red, pero comparte el kernel del sistema anfitrión. Entre las características administrables de un contenedor se encuentran el mapeo de puertos entre el host y el contenedor, la ejecución en segundo plano (modo detached), las políticas de reinicio automático y la posibilidad de asignarle un nombre personalizado para facilitar su identificación.

3.4 Volúmenes

Los volúmenes son mecanismos de persistencia y de intercambio de datos entre el sistema anfitrión y el contenedor. Mediante un volumen, una carpeta del equipo local puede montarse dentro del sistema de archivos del contenedor, de manera que cualquier cambio realizado en los archivos locales se refleje de inmediato en el contenedor sin necesidad de reconstruirlo ni de acceder directamente a su interior. Este mecanismo es especialmente útil para servir contenido web dinámico, como páginas HTML y hojas de estilo CSS, a través de un servidor como Nginx.

3.5 Administración y Ciclo de Vida del Contenedor

Docker provee un conjunto de comandos para administrar el ciclo de vida completo de un contenedor: creación, inicio, detención, reinicio, inspección de procesos y de red, consulta de registros (logs), acceso a su terminal interna y eliminación. Estas operaciones permiten diagnosticar el estado de una aplicación en ejecución y responder ante incidencias sin necesidad de reconstruir el entorno desde cero.

3.6 Redes en Docker

Docker crea automáticamente redes virtuales que permiten la comunicación entre contenedores y con el sistema anfitrión. Cada contenedor puede conectarse a una o varias redes, lo que posibilita, por ejemplo, que distintos servicios se comuniquen entre sí de forma aislada del resto del sistema. La correcta administración de imágenes, contenedores, volúmenes y redes en conjunto es lo que permite construir soluciones de infraestructura reproducibles y escalables mediante Docker.

4. Actividades Realizadas

A continuación, se detallan las actividades desarrolladas durante la ejecución práctica del examen, en el orden establecido por la guía correspondiente.

4.1 Preparación del Entorno

Se verificó el correcto funcionamiento de Docker Desktop y se descargó la imagen oficial de Nginx desde Docker Hub. Se confirmó la descarga exitosa mediante el listado de imágenes disponibles y se eliminaron los contenedores anteriores relacionados con prácticas previas, con el fin de iniciar la actividad en un entorno limpio.

4.2 Implementación del Servidor Web

Se creó el contenedor con el nombre web-lab a partir de la imagen de Nginx, exponiendo el puerto 8080, configurado para ejecutarse en segundo plano y con política de reinicio automático en caso de detenerse. Se verificó que el contenedor quedara en ejecución correctamente mediante el listado de contenedores activos, tanto por la línea de comandos como desde el panel de Docker Desktop.

4.3 Publicación del Sitio Web

Se desarrolló una página HTML personalizada con la información de la universidad, la asignatura, los nombres de los estudiantes, el número de cuenta, el docente y la fecha de entrega, incorporando además un menú de navegación, una imagen institucional, hoja de estilos CSS propia y un pie de página. El sitio se publicó mediante un volumen montado en el contenedor, sin modificar directamente su contenido interno, lo que permitió que cualquier cambio realizado en los archivos locales se reflejara de inmediato en el navegador.

4.4 Administración del Contenedor

Se ejecutaron las acciones de administración solicitadas sobre el contenedor web-lab: detener y reiniciar el contenedor, consultar sus registros (logs), ingresar a su terminal interna, listar los procesos activos, obtener su dirección IP y verificar la red a la que se encuentra conectado, además de eliminarlo y volver a crearlo sin necesidad de descargar nuevamente la imagen.

4.5 Administración General de Docker

Se realizaron tareas generales de administración de Docker: listado de imágenes y de contenedores (activos y totales), eliminación de imágenes no utilizadas, visualización del consumo de recursos del contenedor, información general de Docker, revisión de los volúmenes y redes existentes, así como la creación de un nuevo volumen y la conexión del contenedor a una red.

5. Conclusiones

- Docker permite desplegar servicios funcionales, como un servidor web, sin necesidad de instalar software directamente sobre el sistema operativo anfitrión.
- El uso de volúmenes resulta fundamental para publicar y actualizar contenido web de forma dinámica, manteniendo una separación clara entre los datos y el contenedor que los sirve.
- El dominio de los comandos básicos de administración (creación, inspección, detención, eliminación y recreación de contenedores) es esencial para el manejo eficiente de entornos basados en contenedores.
- La comprensión conjunta de imágenes, contenedores, volúmenes y redes constituye la base para simular, a pequeña escala, situaciones reales de administración de infraestructura en un entorno empresarial.

6.Anexos

```
Windows PowerShell
Copyright (C) Microsoft Corporation. Todos los derechos reservados.

Instale la versión más reciente de PowerShell para obtener nuevas características y mejoras. https://aka.ms/PSWindows

PS C:\Users\josue> docker ps
CONTAINER ID   IMAGE     COMMAND                  CREATED        STATUS        PORTS
NAMES
a574d388809b   nginx    "/docker-entrypoint...." 19 hours ago   Up 7 minutes   0.0.0.0:8080->80/tcp, [::]:8080->80/tcp
web-lab
PS C:\Users\josue> docker start web-lab
web-lab
PS C:\Users\josue>
```

```
PS C:\Users\josue> docker logs web-lab
/docker-entrypoint.sh: /docker-entrypoint.d/ is not empty, will attempt to perform configuration
/docker-entrypoint.sh: Looking for shell scripts in /docker-entrypoint.d/
/docker-entrypoint.sh: Launching /docker-entrypoint.d/10-listen-on-ipv6-by-default.sh
10-listen-on-ipv6-by-default.sh: info: Getting the checksum of /etc/nginx/conf.d/default.conf
10-listen-on-ipv6-by-default.sh: info: Enabled listen on IPv6 in /etc/nginx/conf.d/default.conf
/docker-entrypoint.sh: Sourcing /docker-entrypoint.d/15-local-resolvers.envsh
/docker-entrypoint.sh: Launching /docker-entrypoint.d/20-envsubst-on-templates.sh
/docker-entrypoint.sh: Launching /docker-entrypoint.d/30-tune-worker-processes.sh
/docker-entrypoint.sh: Configuration complete; ready for start up
2026/07/07 18:23:21 [notice] 1#1: using the "epoll" event method
2026/07/07 18:23:21 [notice] 1#1: nginx/1.31.2
2026/07/07 18:23:21 [notice] 1#1: built by gcc 14.2.0 (Debian 14.2.0-19)
2026/07/07 18:23:21 [notice] 1#1: OS: Linux 6.6.114.1-microsoft-standard-WSL2
2026/07/07 18:23:21 [notice] 1#1: getrlimit(RLIMIT_NOFILE): 1048576:1048576
2026/07/07 18:23:21 [notice] 1#1: start worker processes
2026/07/07 18:23:21 [notice] 1#1: start worker process 29
2026/07/07 18:23:21 [notice] 1#1: start worker process 30
2026/07/07 18:23:21 [notice] 1#1: start worker process 31
2026/07/07 18:23:21 [notice] 1#1: start worker process 32
2026/07/07 18:23:21 [notice] 1#1: start worker process 33
2026/07/07 18:23:21 [notice] 1#1: start worker process 34
2026/07/07 18:23:21 [notice] 1#1: start worker process 35
2026/07/07 18:23:21 [notice] 1#1: start worker process 36
2026/07/07 18:23:21 [notice] 1#1: start worker process 37
2026/07/07 18:23:21 [notice] 1#1: start worker process 38
2026/07/07 18:23:21 [notice] 1#1: start worker process 39
2026/07/07 18:23:21 [notice] 1#1: start worker process 40
172.17.0.1 -- [07/Jul/2026:18:24:18 +0000] "GET / HTTP/1.1" 200 1319 "-" Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/149.0.0.0 Safari/537.36 "-"
```

```
Windows PowerShell

/docker-entrypoint.sh: Launching /docker-entrypoint.d/10-listen-on-ipv6-by-default.sh
10-listen-on-ipv6-by-default.sh: info: IPv6 listen already enabled
/docker-entrypoint.sh: Sourcing /docker-entrypoint.d/15-local-resolvers.envsh
/docker-entrypoint.sh: Launching /docker-entrypoint.d/20-envsubst-on-templates.sh
/docker-entrypoint.sh: Launching /docker-entrypoint.d/30-tune-worker-processes.sh
/docker-entrypoint.sh: Configuration complete; ready for start up
2026/07/08 13:49:22 [notice] 1#1: using the "epoll" event method
2026/07/08 13:49:22 [notice] 1#1: nginx/1.31.2
2026/07/08 13:49:22 [notice] 1#1: built by gcc 14.2.0 (Debian 14.2.0-19)
2026/07/08 13:49:22 [notice] 1#1: OS: Linux 6.6.114.1-microsoft-standard-WSL2
2026/07/08 13:49:22 [notice] 1#1: getrlimit(RLIMIT_NOFILE): 1048576:1048576
2026/07/08 13:49:22 [notice] 1#1: start worker processes
2026/07/08 13:49:22 [notice] 1#1: start worker process 22
2026/07/08 13:49:22 [notice] 1#1: start worker process 23
2026/07/08 13:49:22 [notice] 1#1: start worker process 24
2026/07/08 13:49:22 [notice] 1#1: start worker process 25
2026/07/08 13:49:22 [notice] 1#1: start worker process 26
2026/07/08 13:49:22 [notice] 1#1: start worker process 27
2026/07/08 13:49:22 [notice] 1#1: start worker process 28
2026/07/08 13:49:22 [notice] 1#1: start worker process 29
2026/07/08 13:49:22 [notice] 1#1: start worker process 30
2026/07/08 13:49:22 [notice] 1#1: start worker process 31
2026/07/08 13:49:22 [notice] 1#1: start worker process 32
2026/07/08 13:49:22 [notice] 1#1: start worker process 33

What's next:
  View and search logs for all containers in one place
  with Docker Desktop's Logs view. docker-desktop://dashboard/logs
PS C:\Users\josue> docker exec -it web-lab bash
root@a574d388809b:/#
```

```

Windows PowerShell
Copyright (C) Microsoft Corporation. Todos los derechos reservados.

Instale la versión más reciente de PowerShell para obtener nuevas características y mejoras. https://aka.ms/PSWindows

PS C:\Users\josue> docker push eduarjosue83/mi-sitio:2.0
The push refers to repository [docker.io/eduarjosue83/mi-sitio]
tag does not exist: eduarjosue83/mi-sitio:2.0
PS C:\Users\josue> docker images

```

IMAGE	ID	DISK USAGE	CONTENT SIZE	EXTRA
eduarjosue83/mi-sitio:1.0	9d320f63ba48	238MB	63.1MB	
hello-world:latest	96498ffd522e	25.9kB	9.49kB	
mi-sitio:1.0	9d320f63ba48	238MB	63.1MB	
miweb-nginx:latest	1cacf6e07657	92.5MB	26MB	
mysql:latest	e370cd5f6459	1.28GB	287MB	
nginx:latest	ec4ed8b5299e	241MB	66MB	U
tomsik68/xampp:latest	460987e0dc92	1.87GB	491MB	
ubuntu:latest	53958ec7b67c	160MB	45.3MB	

```

PS C:\Users\josue> docker tag eduarjosue83/mi-sitio:1.0 eduarjosue83/mi-sitio:2.0
PS C:\Users\josue> cd "C:\Users\josue\OneDrive\Documentos\EXAMEN I LABORATORIO"
PS C:\Users\josue\OneDrive\Documentos\EXAMEN I LABORATORIO> docker build -t eduarjosue83/mi-sitio:2.0 .
[+] Building 1.9s (9/9) FINISHED
=> [internal] load build definition from Dockerfile
=> => transferring dockerfile: 175B
=> [internal] load metadata for docker.io/library/nginx:latest
=> [internal] load .dockerignore
=> => transferring context: 2B
=> [1/4] FROM docker.io/library/nginx:latest@sha256:ec4ed8b5299e5e90694af7750eb6dffd2627317d30544d056b0371f8082f
=> => resolve docker.io/library/nginx:latest@sha256:ec4ed8b5299e5e90694af7750eb6dffd2627317d30544d056b0371f8082f

```

```

=> [internal] load build context
=> => transferring context: 30.10kB
=> [2/4] COPY index.html /usr/share/nginx/html/
=> [3/4] COPY style.css /usr/share/nginx/html/
=> [4/4] COPY Logo.png /usr/share/nginx/html/
=> exporting to image
=> => exporting layers
=> => exporting manifest sha256:07656ab5606e1f33b087092738fe67c3a6b9a26adf340cb371f72d8914b01ba7
=> => exporting config sha256:92d75dfc66b91072a004438c3ea530b9bf20b9c654c9ac8ebca9fed1090248a9
=> => exporting attestation manifest sha256:030c348213d51fe006359af9374bfe6170978c3e6ee4bfffef7290f361c7de
=> => exporting manifest list sha256:9b45c47073a5da377432fef06b127c1590bf114f0de53b277a6ecd8288fe28f4
=> => naming to docker.io/eduarjosue83/mi-sitio:2.0
=> => unpacking to docker.io/eduarjosue83/mi-sitio:2.0

View build details: docker-desktop://dashboard/build/desktop-linux/desktop-linux/62v19vd7tjb57asui3kwo47a0
PS C:\Users\josue\OneDrive\Documentos\EXAMEN I LABORATORIO> docker push eduarjosue83/mi-sitio:2.0
The push refers to repository [docker.io/eduarjosue83/mi-sitio]
6e32af6cf1ad: Pushed
acf093e7a04f: Layer already exists
1645c1e06f46: Layer already exists
d48d2466050e: Pushed
1b30016634d5: Layer already exists
cd9307c9ecd8: Layer already exists
7908c5a136b9: Pushed
1e7f4bce4e54: Pushed
e95a6c7ea7d4: Layer already exists
df68ee7e7a00: Layer already exists
fcb6fd84b2a0: Layer already exists
2.0: digest: sha256:9b45c47073a5da377432fef06b127c1590bf114f0de53b277a6ecd8288fe28f4 size: 856
PS C:\Users\josue\OneDrive\Documentos\EXAMEN I LABORATORIO>

```